

2026 시스템반도체설계 HW#1

Modular Arithmetic Unit

2026-2

Computer Systems and Reliable SOC Lab.
Yonsei University

Outline

- 이론 배경

- Background
- Modular Arithmetic
 - Modular Arithmetic Implementation
 - Barrett Reduction
- Butterfly 연산

- 과제

- 1. Modular Arithmetic Unit 구현
- 2. Butterfly Core 구현
- Verification
- Submission

Background

- ML-KEM: 2024년 NIST가 표준으로 정한 양자내성 키 캡슐화 메커니즘 (FIPS 203)
 - 256개의 계수를 갖는 다항식을 기본 데이터로 사용
 - 각 계수는 0 ~ 3328 사이의 정수 (12-bit)
- 키 생성 · 캡슐화 · 디캡슐화마다 다항식 곱셈이 반복 수행됨
- NTT 회로는 해시(Keccak) 회로와 함께 ML-KEM 하드웨어의 핵심 블록
 - NTT는 다항식 곱셈 연산량을 $O(n^2)$ 에서 $O(n \log n)$ 으로 줄이는 변환
- HW#1: NTT의 기본 연산 단위인 modular 연산기와 butterfly 연산기 구현

Modular Arithmetic

- $a \bmod q$ (정수 a , 양의 정수 q)
 - a 를 q 로 나눈 나머지 r 을 의미
 - $a = \lfloor a / q \rfloor \times q + r, 0 \leq r < q$
 - 결과는 항상 0 이상 $q-1$ 이하의 정수
- Modular congruence $\equiv$ (합동)
 - $a \equiv b$: a 와 b 를 q 로 나눈 나머지가 같다는 뜻
 - $(a - b)$ 가 q 로 나누어 떨어짐
- Modular 연산 성질
 - 연산 중간 단계에서 reduction하여도 최종 결과는 동일
 - $(a + b) \bmod q = ((a \bmod q) + (b \bmod q)) \bmod q$
 - $(a - b) \bmod q = ((a \bmod q) - (b \bmod q)) \bmod q$
 - $(a \times b) \bmod q = ((a \bmod q) \times (b \bmod q)) \bmod q$
 - → 하드웨어에서는 매 연산마다 reduction하여 bit 폭을 고정함 ($q = 3329 \rightarrow 12\text{-bit}$)

Modular Arithmetic

- Example

- $a \bmod 17$

a	$a = \lfloor a / 17 \rfloor \times 17 + r$	$a \bmod 17$
20	$1 \times 17 + 3$	3
64	$3 \times 17 + 13$	13
100	$5 \times 17 + 15$	15
-1	$(-1) \times 17 + 16$	16

- Modular congruence (합동, $\equiv$)

- $64 \equiv 13 \pmod{17}$: $64 - 13 = 51 = 3 \times 17$
- $-1 \equiv 16 \pmod{17}$: $-1 - 16 = -17 = (-1) \times 17$

Modular Arithmetic

연산	식	결과
+	$(12 + 9) \bmod 17$	4
-	$(3 - 9) \bmod 17$	11
×	$(8 \times 9) \bmod 17$	4

- 중간 reduction 효과: $(64 \times 4) \bmod 17$

최종 reduction

$$64 \times 4 = 256 = 15 \times 17 + 1 \\ \rightarrow 1$$

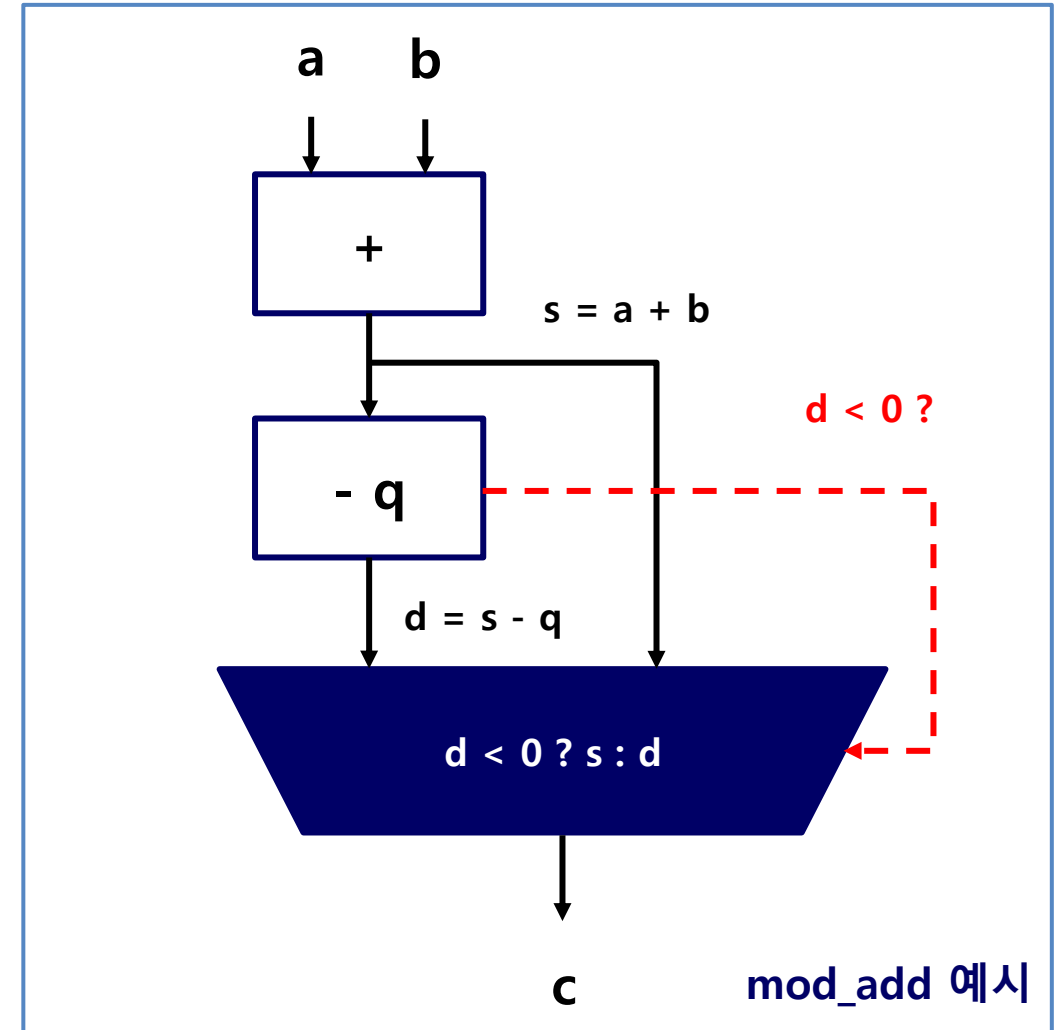
중간 reduction

$$64 \bmod 17 = 13 \\ \rightarrow 13 \times 4 = 52 = 3 \times 17 + 1 \\ \rightarrow 1$$

- 하드웨어는 매 연산마다 reduction하여 중간 값의 bit 폭을 작게 유지함

Modular Arithmetic Implementation

- $a, b \in [0, q-1]$ 가정
- Mod_add
 - $0 \leq a + b \leq 2q - 2$
 - q 이상인 경우 q 를 1회 감산
- Mod_sub
 - $-q + 1 \leq a - b \leq q - 1$
 - 음수인 경우 q 를 1회 가산
- Mod_mul
 - $0 \leq a \times b \leq (q-1)^2$
 - 1회 감산으로 불충분 → Barrett reduction 적용



Barrett Reduction

- Problem

- Mod_mul의 곱의 범위: $0 \leq a \times b \leq (q-1)^2$
 - $q=3329$ 인 경우, $0 \leq a \times b \leq 11,075,584$
 - q 의 1회 감산으로는 reduction 불가능
- $c = p \bmod q = p - q \times \lfloor p / q \rfloor$ ($p = a \times b$)
 - 몫 $\lfloor p / q \rfloor$ 을 구하면 곱셈 1회와 뺄셈 1회로 계산 가능
 - 몫을 구하기 위한 나눗셈 연산은 overhead가 큼

- 몫 $\lfloor p / q \rfloor$ 구하기

- q 로 나누는 연산은 $1/q$ 을 곱하는 연산과 동일함
- $1/q$ 을 정수와 2의 거듭제곱의 비로 근사
 - $1/q \approx \mu / 2^k$, $\mu = \lfloor 2^k / q \rfloor$ (사전 계산 상수)
 - $\lfloor p / q \rfloor \approx (p \times \mu) \gg k$
- → 몫을 구하기 위한 나눗셈 연산을 상수 곱셈 1회와 shift 연산으로 대체

Barrett Reduction

- Example ($q = 13$)

- $k = 8, \mu = \lfloor 2^8 / 13 \rfloor = 19$

- Algorithm

- 1. $p = a \times b$
 - 2. $t = (p \times \mu) \gg k$
 - 3. $r = p - t \times q$
 - 4. $r \geq q$ 이면 $c = r - q$, 아니면 $c = r$

- k값 구하기

- 2^k 가 $a \times b$ 의 최댓값보다 커야 함 ($2^k > (q-1)^2$)
 - 2의 거듭제곱 중 $a \times b$ 의 최댓값을 처음으로 넘는 지점 찾기

- Example

- $q = 13: 2^7 = 128 < 144 < 2^8 = 256 \rightarrow k = 8$

- 더 간단한 규칙

- 입력이 W -bit이면, 곱은 최대 $2W$ -bit $\rightarrow k=2W$ 이면 조건 만족

Case1: $r < q$
 $a = 5, b = 7$

1. $p = a \times b = 35$
2. $t = (35 \times 19) \gg 8 = 2$
3. $r = 35 - 2 \times 13 = 9$
4. $9 < 13, c = r = 9$

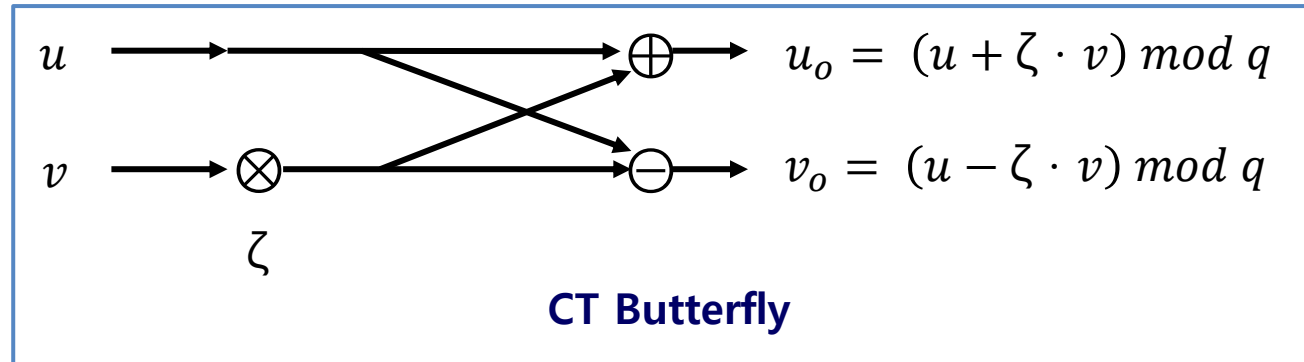
Case2: $r \geq q$
 $a = b = 12$

1. $p = a \times b = 144$
2. $t = (144 \times 19) \gg 8 = 2736 \gg 8 = 10$
3. $r = 144 - 10 \times 13 = 14$
4. $14 \geq 13, c = r - q = 1$

Butterfly 연산

- CT (Cooley-Tukey) Butterfly

- $u_o = (u + \zeta \cdot v) \bmod q$
- $v_o = (u - \zeta \cdot v) \bmod q$



- ζ (zeta, twiddle factor)

- v 에 곱해지는 입력
- 본 과제에서는 값 ($\zeta = 2580$)으로 주어질 예정

- 예시: $q = 17, u = 8, v = 15, \zeta = 4$

신호	계산	결과
$\zeta \cdot v$	$60 = 3 \times 17 + 9$	9
u_o	$(8 + 9) \bmod 17$	0
v_o	$(8 - 9) \bmod 17$	16

1. Modular Arithmetic Unit 구현

- 목표

- $q=3329$ 에 대한 modular adder, subtractor, multiplier를 Verilog로 구현

- 모듈별 기능 및 입출력

모듈명	Input	Output	수행 연산
mod_add	12-bit a, b	12-bit c	$c = (a + b) \bmod q$
mod_sub	12-bit a, b	12-bit c	$c = (a - b) \bmod q$
mod_mul	12-bit a, b	12-bit c	$c = (a \times b) \bmod q$

- 입출력 조건

- 모든 입력과 출력은 unsigned 12-bit
- Input 범위 : $0 \leq a, b < q$
- Output 범위 : $0 \leq c < q$

- 구현 조건

- 각 모듈을 독립적인 combinational 회로로 구현
- $/$, $\%$ 연산자 사용 금지 (Verilog 모듈 구현 시)

2. Butterfly Core 구현

- 목표
 - CT butterfly 구현 (bfly_ct.v)
- 입출력 조건
 - 입력 : u, v, ζ (zeta) (unsigned 12-bit)
 - 출력 : uo, vo (unsigned 12-bit)
 - 입출력 범위 : 0 이상 q 미만
- 구현 조건
 - 1에서 구현한 modular adder, subtractor, multiplier를 활용
 - Combinational 회로로 구현
 - $q=3329$
 - ζ (zeta) = 2580 (testbench에서 입력)

Verification

- Modular Arithmetic Unit과 CT Butterfly의 functionality 검증

- C/Python으로 golden model을 작성하고, 이를 기준으로 Verilog testbench에서 RTL 출력과 기댓값 비교
 - mod_add, mod_sub, mod_mul, bfly_ct를 각각 검증
 - /, % 연산자는 C/Python 및 testbench 검증 시 사용 가능
- 제공된 golden test vector와 별도로, 각 모듈에 대해 corner case를 포함한 서로 다른 입력 조합 20개 이상 검증
 - Corner case 선정 이유 서술

- Modular Arithmetic Unit golden test vector

a	b	mod_add	mod_sub	mod_mul
0	0	0	0	0
123	456	579	2996	2824
987	2100	3087	2216	2062
1664	1665	0	3328	832
1665	1665	1	0	2497
3328	3328	3327	0	1

Verification

- Butterfly Core golden test vector
 - ζ (zeta) = 2580

u	v	uo	vo
0	1	2580	749
749	1	0	1498
123	456	1466	2109
2345	1234	212	1149
1234	2345	2541	3256

Submission

- 제출 파일

- Verilog code (Modular adder / subtractor / multiplier / Butterfly core 모듈, testbench 파일)

- Report

- 회로 기능 및 구현 방법
- 회로의 functionality 검증 방법 및 과정
- ModelSim을 사용한 simulation 결과 (waveform, output 분석)
- Corner case의 입력, 선정 이유, 기댓값, 출력 설명
- Golden model 코드, testbench, 전체 검증 결과 분석

- 감점 요인

- 늦은 제출
- 필수 제출 파일 누락
- 구현 조건 미준수
- copy code는 0점 부여

- Due date

- 10/18 (일) 23:59
- 학번_이름.zip으로 런어스 HW1 제출란에 제출